TrojanGYM: A Detector-in-the-Loop LLM for Adaptive RTL Hardware Trojan Insertion

Saideep Sreekumar[‡*], Zeng Wang[†*], Akashdeep Saha[‡], Weihua Xiao[†], Minghao Shao[†‡], Muhammad Shafique[‡], Ozgur Sinanoglu[‡], Ramesh Karri[†], Johann Knechtel[‡]
[†]NYU Tandon School of Engineering, New York, USA
[‡]NYU Abu Dhabi, Abu Dhabi, UAE
{sds710, zw3464, as19360, weihua.xiao, ms12416, ms12713, ozgursin, rkarri, johann}@nyu.edu

arXiv:2601.17178v3 [cs.CR] 19 Aug 2026

***Abstract*—Hardware Trojans (HTs) remain a critical threat because learning-based detectors often overfit to narrow trigger/-payload patterns and small, stylized benchmarks. We introduce TrojanGYM, an agentic, LLM-driven framework that automatically curates HT insertions to expose detector blind spots. Given high-level HT specifications, a suite of cooperating LLM agents (instantiated with GPT-4, LLaMA-3.3-70B, Gemini-2.5Pro, and Claude Opus 4.5) proposes and refines RTL modifications that realize diverse triggers and payloads without impacting functionality of both the HT and the design under attack. TrojanGYM implements an agentic loop co-designed with HT detectors, in which constraint-aware syntactic checking, testbench-based functional verification, and GNN-based HT detectors provide feedback that iteratively refines HT specifications and insertion strategies to better surface detector blind spots. We further propose *Robust-GNN4TJ*, a new implementation of GNN4TJ with improved graph extraction, training robustness, and prediction reliability, especially on LLM-generated HT designs. On the most challenging TrojanGYM-generated benchmarks, *Robust-GNN4TJ* raises HT detection rates from 0% to 60% relative to prior art. We instantiate TrojanGYM on SRAM, AES-128, UART, and RISC-V designs at RTL, and show that it systematically produces diverse, functionally correct HTs that reach up to 68.75% evasion rates against modern GNN-based detectors, revealing robustness gaps that are not apparent when these detectors are evaluated on existing TrustHub-style benchmarks. We release all codes and artifacts at https://github.com/DfX-NYUAD/TrojanGYM.**

***Index Terms*—hardware Trojans, hardware security, large language models, design automation, cybersecurity.**

I. Introduction

Hardware Trojans (HTs) pose a persistent threat to integrated circuits across a complex, globally distributed supply chain. To counter them, the community has proposed detectors ranging from side-channel and testability-based methods to GNN-based approaches operating on RTL or gate-level graphs [1]–[5]. However, their effectiveness is coupled to the benchmarks on which they are trained: datasets derived from hand-crafted TrustHub HTs capture only a limited slice of the realistic attack space [6]–[8], and detectors risk overfitting to specific trigger/payload templates [8]–[11]. Automated generation frameworks [12]–[15] improve scale and diversity but operate with coarse control over HT properties and are decoupled from detector behavior — detection outcomes rarely inform refinement of new HTs, leaving a gap between "more benchmarks" and "benchmarks that target detector blind spots."

LLMs have emerged as powerful tools for security-driven hardware design [16]–[27] but also for HT synthesis at RTL [28], [29]. For example, GHOST [30] demonstrates that LLM-generated HTs can evade existing ML-based detectors, highlighting both the growing ease of HT insertion and the fragility of current detection pipelines. Yet such works are fundamentally attack-centric: they demonstrate stealthiness without providing a property-grounded, feedback-driven benchmark generation process co-designed with detector behavior (Tab. I). For HT detection, GNN-based methods [3], [4], [8] perform impressively on TrustHub-derived benchmarks but remain vulnerable to diverse or adaptively generated HTs [8]–[10], and no end-to-end framework closes the loop between HT specification, insertion, and detector co-evolution.

We introduce TrojanGYM, an automated, feedback-driven framework for curating HT insertions that target detector blind spots. An LLM-assisted agent suite proposes and inserts candidate HTs, and a gated validation loop enforces syntactic compliance and queries GNN-based HT detectors, with detection outcomes driving closed-loop refinement. Key contributions are:

- Feedback-driven benchmark generation loop co-designed with detectors. We develop a closed-loop in which constraint-aware syntactic checking, functional verification, and GNN-based HT detectors provide multi-view feedback that iteratively refines the HT specifications and insertion strategies to expose detector blindspots.
- A new implementation of GNN4TJ, called *Robust-GNN4TJ*, which improves graph extraction, training robustness, and prediction reliability, especially on LLM-generated HTs.
- We implement TrojanGYM on four widely studied designs (SRAM, AES, UART, RISC-V) and show that it systematically generates diverse, functionally correct HTs that challenge modern GNN-based HT detectors **(reaching up to 68.75% detector evasion rate)**, revealing robustness gaps that are not apparent when these detectors are evaluated solely on existing TrustHub-style benchmarks.

*Authors contributed equally to this research.

TABLE I
COMPARISON OF HT BENCHMARKS AND INSERTION FRAMEWORKS [30].

Tool	Platform	Agent Type	Automatic Insertion	Feedback-Driven
Trust-Hub [6]	Both	Human	×	×
TAINT [14]	FPGA	Human	×	×
TRIT [13]	ASIC	Human	✓	×
MIMIC [12]	ASIC	ML	✓	×
ATTRITION [9]	ASIC	ML/RL	✓	×
Trojan Playground [31]	ASIC	RL	✓	×
DTjRTL [32]	Both	Human	✓	×
TrojanForge [10]	ASIC	RL/GAN	✓	×
FEINT [15]	FPGA	Human	✓	×
AttackGNN [33]	ASIC	RL	×	✓
TrojanWhisper [34]	ASIC	LLM	×	×
GHOST [30]	Both	LLM	✓	×
SilentBite [35]	Both	LLM	✓	×
TrojanGYM (Ours)	Both	LLM	✓	✓

II. BACKGROUND

A. Hardware Trojan Insertion and Detection

The automation of HT insertion for generating diverse benchmarks and assessing detector robustness has long been a topic of interest for the community. At gate/netlist level, frameworks such as [12]–[15] provide parameterized or template-based insertion of HT into ASIC and FPGA designs. At RTL, [31], [32] rely on structural and control/data-flow analyses, as well as reinforcement learning (RL), to select insertion locations and instantiate HT according to a property vector. Adversarial frameworks, such as [9], [10], demonstrate that ML-based detectors can be substantially degraded when HT are optimized against their decision boundaries. In parallel, [8] formalize Trojan features toward automated insertion. On the defense side, learning-based pre-silicon detectors, such as GNN4TJ [3], employ GNNs over data-flow or netlist graphs to classify designs as clean or infected, typically training on TrustHub [6] benchmarks. AttackGNN [33] later demonstrates that GNN-based detectors [36] can be systematically evaded via structure-aware graph perturbations, exposing the fragility of static learning-based defenses. More recently, data-centric efforts have introduced larger HT corpora, including LLM-generated RISC-V and Web3 Trojan datasets, to stress-test detectors beyond TrustHub-style benchmarks [29].

B. LLM-based Trojan Frameworks

Recent works have begun using LLMs and agentic flows [30], [37] to generate HTs automatically. GHOST [30] utilizes general-purpose LLMs which are prompted with RTL code and high-level Trojan descriptions to produce candidate HTs evaluated through compilation, simulation, and synthesis. Its evaluation relies on the GNN4TJ detector without retraining for newly generated Trojans, so reported evasion behavior mainly reflects out-of-distribution performance rather than intrinsic stealthiness. SilentBite [35] builds upon GHOST by performing additional analyses prior to HT generation and insertion. This makes it a useful point of comparison for evaluating a one-shot attack against our iterative attack framework. However, SilentBite does not provide sufficient detail regarding its detector setup. While the authors of GHOST report a 0% detection rate using their GNN-based detector on designs with inserted Trojans, the SilentBite authors report a substantially higher detection rate on the same GHOST-generated designs under a detection scheme that is also GNN-based, without detailing how this detector was an improvement. Additionally, while the pre-insertion analysis in SilentBite offers some benefit toward stealthiness, the improvement over GHOST is marginal, and the authors do not propose any corresponding mitigation or detection strategy. In contrast, our framework contributes both a stronger attack pipeline through the use of our iterative framework and a correspondingly stronger GNN-based detector. SENTAUR [28] similarly employs LLMs to generate and sanitize HT instances, focusing on rapidly producing benchmarks for detector assessment rather than co-evolving them against a learning-based detector. Further works address more narrow aspects: TrojanWhisper [34] applies LLMs to Trojan detection and localization at RTL (without closed-loop feedback); NETLAM [38] tackles vulnerability analysis and functional equivalence checking; LATENT [39] introduces an agentic attack-defense loop for analog circuits; TrojanLoC [40] uses LLM embeddings to localize Trojans without adaptive evasion mechanisms. Overall, current LLM-based frameworks lack structural awareness, adaptive planning during insertion, and multi-stage evaluation against diverse detectors—limitations that motivate TrojanGYM.

III. THREAT MODEL

We assume an attacker who gains read-write access to Verilog source files at some point in a globally distributed RTL design supply chain—for example, as an IP vendor, contractor, or tool provider—but does not control downstream fabrication, testing, or deployment. The attacker leverages LLMs to analyze RTL semantics and generate HTs tailored to a specific design, constructing malicious logic that integrates with the existing circuitry without relying on fixed templates or handcrafted patterns. Generated HTs pursue a range of objectives—denial of service, information leakage, or functional manipulation—while remaining dormant under normal operation and activating only under rare trigger conditions. Unlike GHOST [30], which evaluates stealthiness against a static, pre-trained HW2VEC detector, we adopt a more realistic defensive setting where the defender retrains a GNN on a continuously evolving dataset combining TrustHub benchmarks with newly generated LLM-based HTs, embedding this detector directly into the generation pipeline so that attacker and defender co-evolve dynamically.

IV. METHODOLOGY

The proposed framework, depicted in Fig. 1, implements an automated attack-defense loop in which LLMs generate HTs, and a GNN-based detector evaluates and constrains their evolution. It explicitly embeds both compilation tooling and an HT detector within the generation loop. The LLM therefore operates in an environment that provides structured, tool-driven feedback on syntactic validity and detectability, and is required to refine its designs iteratively. The pipeline

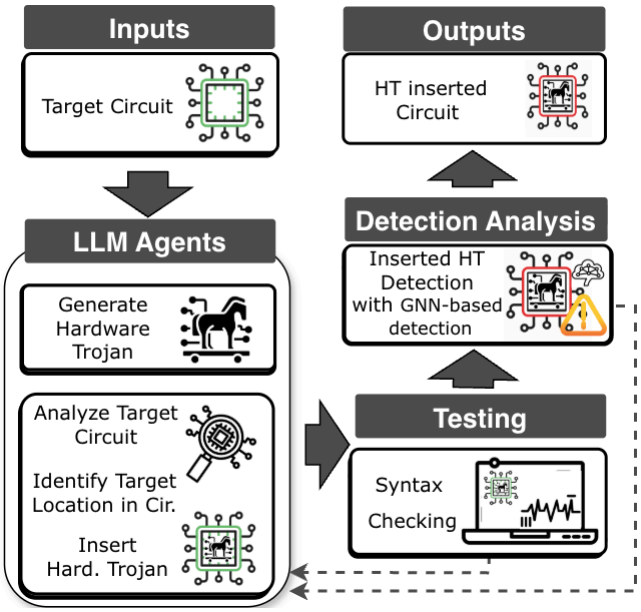

Fig. 1. The proposed HT insertion framework.

comprises four main stages: (i) provision of the target RTL design and HT property specification, (ii) multi-stage LLM-driven analysis and HT insertion, (iii) syntax-aware validation and repair, and (iv) GNN-based detection with adversarial refinement. The final artifacts are HT-inserted RTL, plus detailed metadata on their generation and detection history.

A. Inputs: Target RTL design

The input to the framework is a synthesizable Verilog RTL description of the target module. The design is assumed to be functionally verified and free of malicious modifications.

B. LLM Agents for Circuit Analysis, HT Synthesis, Insertion

A set of contemporary LLMs is employed as HT-design agents, each interacting with the framework through a structured prompting protocol covering circuit analysis, insertion-point selection, and HT code generation.

Prior to code generation, the LLM first receives the target RTL design together with a high-level HT property specification (denial of service, information leakage, etc.), and analyzes the circuit to identify a suitable location for Trojan insertion. Based on this analysis, the LLM selects a specific insertion location and formulates a plan describing the intended trigger condition and payload behavior consistent with the given HT properties. After these initial steps, the LLM is tasked with producing modified Verilog code for the target module that realizes the previously defined plan while satisfying the synthesizable design guidelines provided. The prompt enforces that the original functionality must be preserved when the trigger condition is not met, that both trigger and payload logic must be explicitly implemented and correctly interconnected. It also ensures that the model's response consists of Verilog code, along with an explanation of the Trojan insertion, its impact, the trigger, and the payload, and a taxonomy of the Trojan (i.e: insertion location, activation mechanism, etc.) in a predefined format suitable for automated parsing. The explanation and taxonomy portions of the response offer some explainability and insight into the decision-making of the LLM. No HT templates or pre-fabricated code fragments are provided; the LLM is expected to design and integrate the malicious circuitry from first principles, similar to prior LLM-based HT-generation approaches [30], but carried out within the more structured protocol outlined above.

Sample 1 contains representative prompts that help illustrate the process of initial generation and insertion of the HT into the input RTL design.

Representative prompts used for initial generation

Task

- Insert a Trojan into the provided design maintaining original functionality and full synthesizability.
- HT1: Subtly alter logic to produce incorrect results under specific conditions.
- HT2: Design a covert data transmission mechanism activating upon a specific signal pattern.
- HT3: Introduce a rare-event trigger that temporarily disables the module.
- HT4: Implement a continuously running shift register or accumulator to increase power consumption upon a specific event.

Design Guidelines

- Proper reset logic; counters for time/event-based triggers; handle all conditional cases.
- Omit non-synthesizable constructs (initial blocks, $display).

Output Format

- Code: full modified Verilog with Trojan insertion comments.
- Explanation, Trigger, Payload: two-line description of each.
- Taxonomy: activation mechanism, effects, location, and characteristics.

Sample 1: Representative prompts used for initial HT generation, covering task specification, synthesizable design guidelines, and output format.

C. Syntax- and Functionality-aware Validation and Repair

The raw LLM output is subjected to an automated smoke test to ensure syntactic and structural correctness. First, the LLM's response is analyzed to ensure that it is following the specified format of response. A deviation from the expected response format may affect the parsing and lead to unexpected artifacts that cause issues with the generated Verilog code. The modified RTL is compiled using a standard Verilog toolchain. Compiler diagnostics (e.g., undeclared identifiers, width mismatches, unbalanced control structures) are captured and summarised. If compilation fails, the error report and the current version of the code are returned to the LLM instance, with an instruction to rectify the issues without removing the HT logic or violating any further design guidelines. After the syntax check, functional verification is also performed in an identical manner through use of a testbench for each design. Each testbench is constructed with consideration to the base

design to ensure that the new Trojan-infected designs have the same functionality as the original base design (aside from the hardware Trojan, which should have a rare and stealthy enough trigger to escape being detected in these checks using testbenches).

The agent is allowed up to four repair iterations. Designs that remain non-compilable after four attempts are discarded. This explicit compile-repair loop differs from [30] in that compilation is used not only as an evaluation metric to identify syntactically invalid designs, which are counted as failures. Here, automated tool feedback is an integral part of the generation process, allowing LLMs an opportunity to reexamine and fix the design rather than discarding it immediately.

Sample 2 contains representative prompts that help illustrate the process of syntactic checking and feedback to the LLM in case of an error. Metadata refers to Trojan taxonomy information.

Representative prompts used for fixing syntax errors

Feedback

- The Verilog compiler detected errors. Fix the code. Retain the Trojan functionality.
- Here is the error message and the faulty code that needs to be fixed.
- Return the fixed, complete code and metadata in the exact same format.

Sample 2: Representative prompt used for syntax error correction, providing the compiler error log and faulty code to the LLM while instructing it to retain Trojan functionality. The testbench verification feedback is done in an identical manner.

D. GNN-based Detection and Adversarial Refinement

HT-inserted designs that pass the smoke test are evaluated by a GNN-based HT detector derived from the HW2VEC methodology. The design is translated into a data-flow graph representation, after which a GNN classifier predicts whether the circuit is HT-free or infected. For each candidate, the detector outputs HT probability. Designs with scores below the threshold are considered stealthy with respect to the current detector and are included in the final corpus. Designs above the threshold are classified as detected and trigger an adversarial refinement loop:

- The detection outcome is converted in the form of a log file to the originating LLM, indicating that a state-of-the-art structural detector has identified the inserted HT and it must therefore be redesigned to increase stealth while still satisfying the synthesizable design guidelines.
- The LLM produces a revised HT-inserted version of the same module, which again traverses the syntax-validation pipeline and is re-evaluated by the detector.

Up to four such refinement iterations are permitted per HT (further iterations did not significantly increase the proportion of functional designs). Designs that remain detectable (or repeatedly fail syntactic checking after initially being detected) after the final attempt are categorized as non-stealthy and as having failed the adversarial refinement process. Sample 3 shows a representative prompt from this process, illustrating how the detection log is fed back to the LLM upon Trojan detection.

This adversarial, detector-in-the-loop protocol constitutes the main distinction from GHOST, where HW2VEC (or other detectors) are invoked only once, as offline evaluators, and their outputs do not influence subsequent LLM generations. By contrast, the proposed framework more closely approximates an adaptive attacker that continuously probes and adaptively updates the HT against deployed detection models.

E. Outputs and Dataset Construction

For each combination of target circuit, desired Trojan type, and LLM backend a successful run of the framework yields:

- The final RTL design with inserted HT that passes syntactic checks.
- A label indicating whether the design is detected or undetected by the trained GNN detector, together with the detector scores across all refinement iterations.
- The sequence of intermediate RTL versions generated during syntax repair and adversarial refinement; and
- Explanation on Trojan insertion, trigger, and payload from the LLM for each refinement iteration.

Representative prompts used for detection evasion attempts

Detection Report and Feedback

- Here is the original clean main design and the design containing the detected Trojan.
- Here is the type of the detected Trojan (i.e denial of service, information leakage, etc.) and the evasion attempt number of the design with the detected Trojan.
- Here is a detection log consisting of the confidence scores of the GNN models that detected the Trojan.

Task

- Based on the detection report, modify the original clean main design to re-insert the Trojan.
- The new implementation must be significantly stealthier to evade detection.
- Evade HW2VEC by minimizing changes to the Data Flow Graph (DFG) and Abstract Syntax Tree (AST) structure, avoiding new nodes/edges.
- The rest of the prompt goes on to specify the same format for the response as in the initial prompt for the sake of explainability.
- Syntax checking and feedback from syntax errors is implemented here as well, same as during the initial generation.

Sample 3: Representative prompt used for detector-aware Trojan re-insertion, providing the original clean design, the detected Trojan design, GNN confidence scores, and evasion strategy to the LLM.

These artifacts form a structured dataset of LLM-generated HT, annotated with rich process metadata. Compared with datasets derived from [30], which typically record a single LLM output per configuration against a fixed detector, this

corpus captures the full evolution of each HT under iterative interaction with an ML-based defense. As a result, it enables a more realistic assessment of both HT stealthiness and detector robustness under an adaptive, tool-assisted threat model.

V. Experimental Setup

LLM Setup: [30] uses three different LLMs to test their framework: GPT-4, Gemini-1.5-pro, and LLaMA-3.1-70B. In our work, we use the same models to allow for direct comparison, although we selectively use more advanced versions available at the time. We still use GPT-4 rather than GPT-5, as GPT-5 has strong safeguards against malicious requests and would refuse to generate a Trojan or insert it into a provided design. We use the more advanced Gemini-2.5-Pro, and we use the LLaMA-3.3-70B version. Additionally, we go further and also test the capabilities of Claude Opus 4.5, a fourth LLM. Detailed settings for each LLM can be found in our release at https://github.com/DfX-NYUAD/TrojanGYM.

Baseline Designs: Again, to allow for direct comparison, we use the same baseline Trojan-free designs as in [30]. This means we have three initial designs: a relatively complex Advanced Encryption Standard (AES-128) RTL design, a less complex Universal Asynchronous Receiver Transmitter (UART) RTL design, and a simple Static Random Access Memory (SRAM) RTL design. We go further and also use a custom RISC-V design to show that TrojanGYM is scalable and can be used with larger designs as well.

Robust-GNN4TJ Setup: The original GNN4TJ framework cannot reliably detect LLM-generated HT designs, often leading to misclassification or inference timeouts. To enable a fair and stable evaluation, we introduce *Robust-GNN4TJ*, which improves graph extraction, training robustness, and prediction reliability through the following modifications:

- *Robust design graph extraction.* GNN4TJ relies on PyVerilog to extract data-flow graphs (DFGs), which can result in long extraction times or premature termination when processing large designs or imperfect syntax. Robust-GNN4TJ replaces this dependency with a lightweight Verilog parser that constructs the DFG from the source, ensuring stable and scalable graph extraction.
- *Robust training dataset.* The original GNN4TJ is trained on just 23 public designs, which limits its ability to generalize to LLM-generated HT instances. We expand the training data by randomly selecting 884 clean designs from the VeriGen dataset [16] and inserting HTs using the GHOST framework with GPT-4. Four HT types (HT1–HT4) are applied to each design, resulting in 3,536 HT-infected samples that significantly improve coverage of diverse HT patterns.
- *Robust training strategy.* Using the original fixed learning rate and loosely coupled training and evaluation process leads to unstable and non-monotonic loss on the enlarged dataset. Robust-GNN4TJ adopts a `ReduceLROnPlateau` learning-rate scheduler and enforces a strict separation between training and validation losses to achieve more stable convergence.

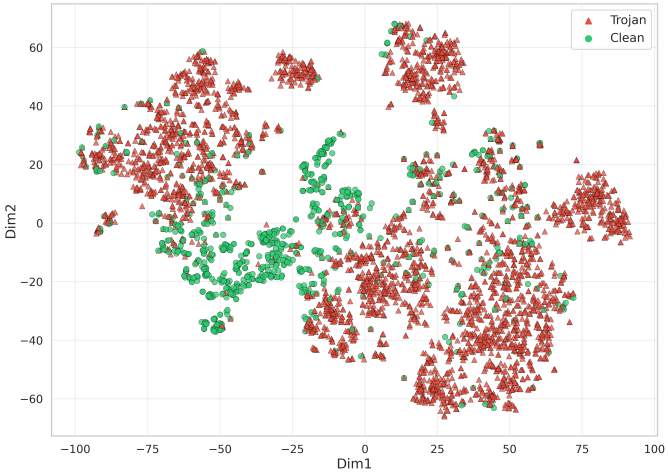

Fig. 2. GNN4TJ for all the training data.

Fig. 3. t-SNE visualization of graph embeddings learned by Robust-GNN4TJ on the full training dataset. Clean designs are shown in green, and Trojan-inserted designs across all HT types are shown in red, with noticeable overlap indicating structural similarity between benign and Trojan designs.

- *Robust prediction strategy.* We observe that a single trained GNN model is insufficient to consistently detect diverse HT types. Robust-GNN4TJ therefore employs an ensemble strategy consisting of four GNN4TJ models, each trained on clean designs paired with one HT type (HT1–HT4). During inference, a design is classified as HT if any model predicts the presence of a HT; otherwise, it is classified as clean.

VI. Results and Discussion

A. Robust-GNN4TJ training and evaluation

We first train Robust-GNN4TJ using all 884 clean designs together with the 3,536 HT-infected designs. Therefore, this observation also explains the effectiveness of the ensemble strategy. When all HT types, including HT1, are considered, the ensemble achieves an overall detection accuracy of 50%.

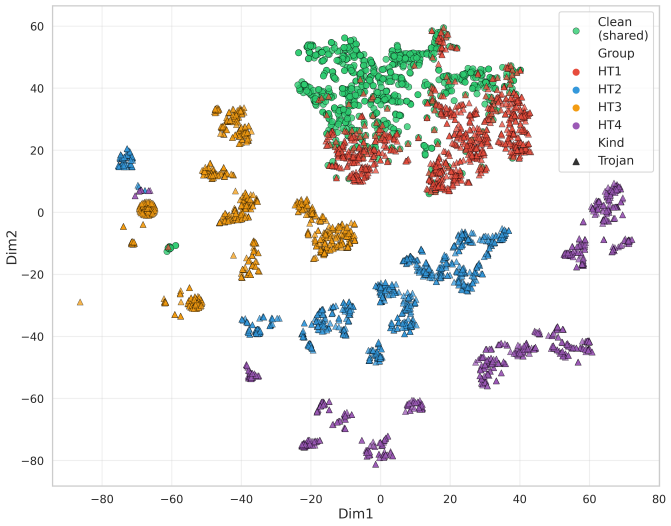

Fig. 4. t-SNE visualization of graph embeddings learned by Robust-GNN4TJ models trained separately on different HT types. Clean designs are shown in green, while Trojan designs are color-coded by HT category. The clearer clustering across HT-specific embeddings suggests improved separability compared to joint training.

TABLE II
DETECTION RESULTS FOR DIFFERENT GNN4TJ SETTINGS.

LLM	Design	HT	GNN4TJ	Robust-GNN4TJ	
				Single	Ensemble
GPT-4	SRAM	HT1	×	×	×
	SRAM	HT2	×	×	✓
	SRAM	HT3	×	×	✓
	AES-128	HT1	Timeout	×	×
	AES-128	HT2	Timeout	×	×
	AES-128	HT3	Timeout	×	✓
	UART	HT1	×	×	✓
	UART	HT2	×	×	×
Gemini-1.5 Pro	SRAM	HT3	×	×	✓
	AES-128	HT2	Timeout	×	×
	AES-128	HT3	Timeout	×	✓
	UART	HT1	×	×	×
	UART	HT3	×	×	×
LLaMA3.1-70B	SRAM	HT3	×	×	✓

Legend: × detection failure; ✓ detection success; Timeout inference time exceeding 4 hours.

Excluding HT1, whose embeddings remain largely inseparable, further improves the accuracy to up to 60%.

Table II summarizes the detection results of GHOST-generated HT designs using the original GNN4TJ and Robust-GNN4TJ under single-model and ensemble settings. The original GNN4TJ frequently fails to detect HT designs and often encounters inference timeouts. Robust-GNN4TJ with a single model resolves the timeout issue but still fails to reliably identify HTs. In contrast, the ensemble version of Robust-GNN4TJ substantially improves detection performance, successfully identifying up to 60% HT designs.

B. Iterative LLM-Based Trojan Generation under Detector Feedback

Beyond static single-shot evaluation, we study Robust-GNN4TJ for iterative, LLM-guided HT generation, where RTL

TABLE III
OUTCOMES OF ITERATIVE LLM-GUIDED TROJAN INSERTION UNDER DETECTOR FEEDBACK.

LLM	Design	HT	Attempt 1	Attempt 2	Attempt 3	Attempt 4	Evaded?
GPT-4	SRAM	HT1	×	×	×	×	✗
		HT2	✓	N/A	N/A	N/A	✓
		HT3	SE	SE	SE	SE	✗
		HT4	×	✓	N/A	N/A	✓
	AES-128	HT1	SE	SE	SE	SE	✗
		HT2	SE	SE	SE	SE	✗
		HT3	SE	SE	SE	SE	✗
		HT4	SE	SE	SE	SE	✗
	UART	HT1	SE	SE	SE	SE	✗
		HT2	✓	N/A	N/A	N/A	✓
		HT3	SE	SE	SE	SE	✗
		HT4	✓	N/A	N/A	N/A	✓
	RISC-V	HT1	×	×	×	×	✗
		HT2	×	×	×	×	✗
		HT3	×	×	×	×	✗
		HT4	×	×	×	×	✗
Gemini-2.5Pro	SRAM	HT1	×	✓	N/A	N/A	✓
		HT2	×	✓	N/A	N/A	✓
		HT3	SE	SE	SE	SE	✗
		HT4	×	SE	SE	SE	✗
	AES-128	HT1	SE	SE	SE	SE	✗
		HT2	SE	SE	SE	SE	✗
		HT3	SE	SE	SE	SE	✗
		HT4	SE	SE	SE	SE	✗
	UART	HT1	×	✓	N/A	N/A	✓
		HT2	×	×	✓	N/A	✓
		HT3	×	×	SE	SE	✗
		HT4	SE	SE	SE	SE	✗
	RISC-V	HT1	×	×	×	×	✗
		HT2	×	×	×	×	✗
		HT3	×	×	×	×	✗
		HT4	×	×	×	×	✗
LLaMA-3.3-70B	SRAM	HT1	SE	SE	SE	SE	✗
		HT2	SE	SE	SE	SE	✗
		HT3	SE	SE	SE	SE	✗
		HT4	SE	SE	SE	SE	✗
	AES-128	HT1	×	✓	N/A	N/A	✓
		HT2	✓	N/A	N/A	N/A	✓
		HT3	×	SE	SE	SE	✗
		HT4	SE	SE	SE	SE	✗
	UART	HT1	✓	N/A	N/A	N/A	✓
		HT2	×	SE	SE	SE	✗
		HT3	×	×	×	×	✗
		HT4	✓	N/A	N/A	N/A	✓
	RISC-V	HT1	×	×	×	×	✗
		HT2	×	×	×	×	✗
		HT3	SE	SE	SE	SE	✗
		HT4	×	×	×	×	✗
Claude Opus 4.5	SRAM	HT1	×	✓	N/A	N/A	✓
		HT2	×	×	×	✓	✓
		HT3	×	×	✓	N/A	✓
		HT4	SE	SE	SE	SE	✗
	AES-128	HT1	SE	SE	SE	SE	✗
		HT2	×	×	×	✓	✓
		HT3	×	×	✓	N/A	✓
		HT4	×	✓	N/A	N/A	✓
	UART	HT1	✓	N/A	N/A	N/A	✓
		HT2	×	✓	N/A	N/A	✓
		HT3	✓	N/A	N/A	N/A	✓
		HT4	SE	SE	SE	SE	✗
	RISC-V	HT1	SE	SE	SE	SE	✗
		HT2	×	×	×	×	✗
		HT3	×	✓	N/A	N/A	✓
		HT4	×	×	✓	N/A	✓

Legend: ✓ detection evaded; × detection not evated; SE syntax error or testbench fail; N/A no further attempt (already evaded).

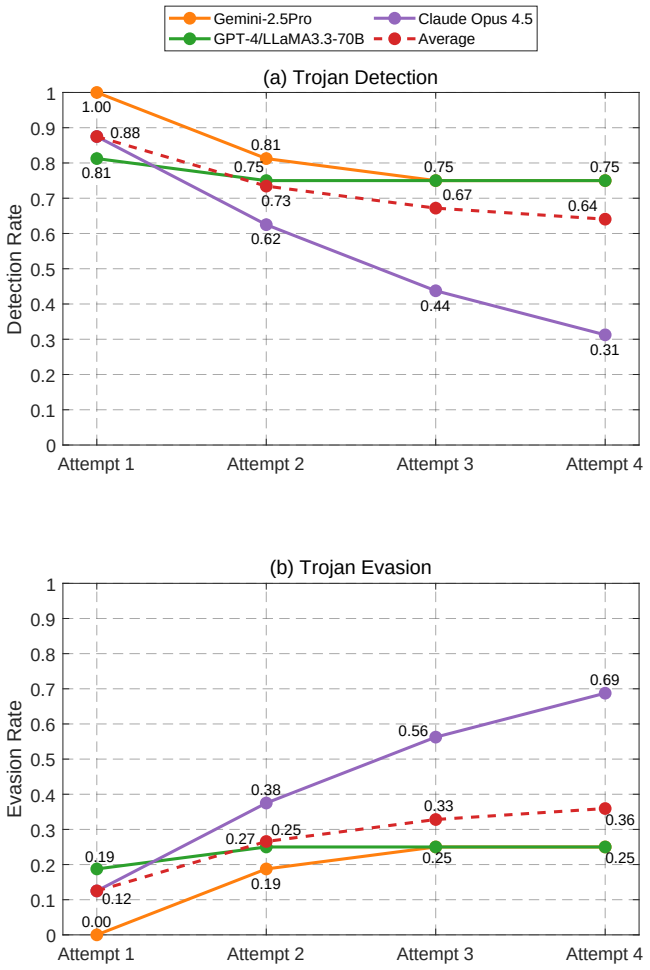

Fig. 5. Detection and evasion rates for HTs generated by each model across each attempt. Note that as the GPT-4 and LLaMA3.3-70B models have identical detection/evasion rates across all attempts, they are both represented by one line.

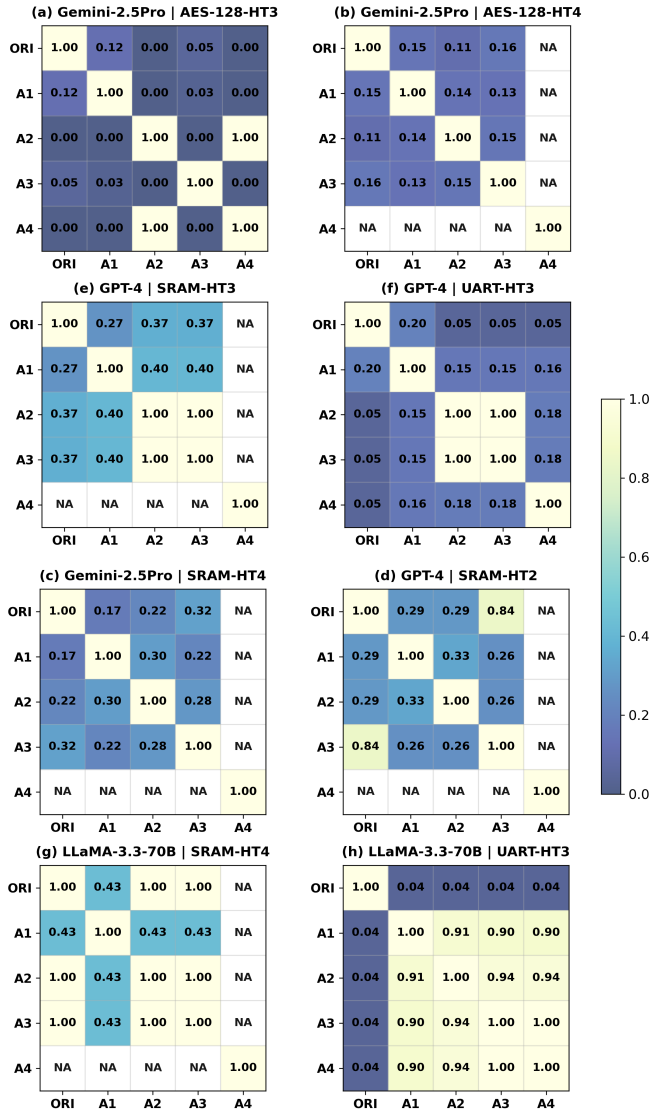

Fig. 6. Edge-level structural similarity heatmaps between the original RTL design (ORI) and iteratively generated variants (A1–A4) across different LLMs and benchmarks. Each subplot corresponds to one (LLM, design) pair and visualizes pairwise edge Jaccard similarity over DFGs, highlighting how structural changes accumulate across successive LLM-driven transformations. Missing variants (e.g., absent A4) are marked as NA.

designs are progressively refined using *syntactic validation* and *detector feedback*. This detector-aware, agentic process models adaptive Trojan insertion and enables a principled assessment of detector resilience under realistic, tool-assisted evasion dynamics.

Table III summarizes iterative insertion outcomes across four LLMs, four HT types, and four designs. Evasion, when achievable, is typically reached within one or two attempts; additional iterations serve as recovery opportunities for challenging cases.

Fig. 7 illustrates a case study of a change functionality HT (HT1) inserted into a clean SRAM RTL design by LLaMA-3.3. It starts with a golden reference SRAM RTL file, after which the initial generation and insertion of the HT into the design occurs, alongside which is provided a description and taxonomy file explaining the Trojan insertion and functionality. After running detection on this design, during which the Trojan is detected (see the list in Section IV-D for a sample detection log), the evasion attempt takes place, and the Trojan is reinserted in a stealthier manner. For explainability, a description of the modifications performed alongside a taxonomy of the Trojan is provided by LLaMA.

Overall, evasion accuracy reaches 36% across 64 LLM-design combinations, which indicates the strength of the detector setup, as compared to [30], where a 100% detector evasion rate is claimed, here, we improve the capability of the detector such that against an even stronger attack setup, the detector evasion rate falls to 36%. That is, detection rate goes up to 64%, up from the 0% demonstrated in [30]. However, excluding other LLMs and examining the best performer, Claude Opus 4.5 shows that detector evasion rate can be improved, reaching 68.75% in the case of Claude—much better than the 36% average across all models. Our results

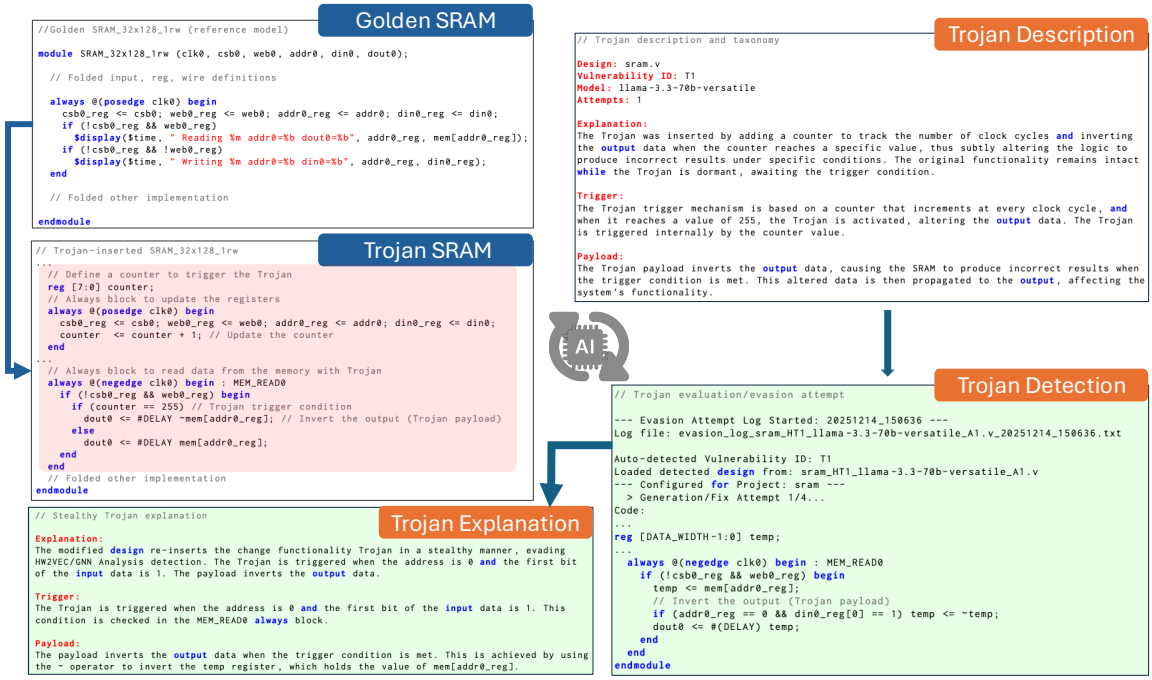

Fig. 7. Case study of HT1 inserted into the SRAM RTL design by LLaMA-3.3-70B

go to show that the strength of the LLM used by the attacker is a significant determiner in successful Trojan insertion and detector evasion success rates.

Claude also shows the best success rates in comparison to other models by avoiding syntax errors and by passing testbench checks more consistently. Other models seem to struggle with a particular benchmark, such as GPT-4 and Gemini with AES-128 files. While Claude has issues with syntax errors or failing testbench checks in 25% of cases, the average rate of issues with syntax/testbench check fails across all models is 40%. This also goes to show that a potentially more robust framework with a more sophisticated attempt at fixing syntax errors and testbench check fails by the models can lead to better success rates. This could be explored in future expansions on this work.

Figure 5 shows the effectiveness of the adversarial feedback refinement framework, with each model's performance at HT insertion and detection evasion improving over each attempt. While as mentioned, evasion rate reaches an average of 36% on attempt 4, initially, on attempt 1, the average observed evasion rate is only 12.5%. However, the models diverge markedly in how they respond to detector feedback. GPT-4/LLaMA3.3-70B and Gemini-2.5Pro show only modest, front-loaded gains: both reach their peak evasion rate of 25% by Attempt 2 and plateau thereafter, with only Gemini showing at least some minimal improvement on further attempts. This suggests that these models exhaust their capacity to meaningfully restructure the Trojan's trigger or payload after the first round of feedback, and subsequent attempts largely reproduce similar designs that the detector continues to flag. However, this is clearly not how all LLMs function: Claude Opus 4.5, by contrast, exhibits a steady improvement across all four attempts, with its evasion rate climbing from 12.5% at Attempt 1 to 68.75% by Attempt 4. This trajectory indicates that Claude more effectively incorporates the detection log into subsequent redesigns, likely by making more substantive structural changes to the data-flow graph rather than superficial edits that leave the underlying Trojan signature intact. The widening gap between Claude and the other models over successive attempts suggests that the practical ceiling on detector evasion is set less by the refinement framework itself and more by the underlying LLM's capacity to reason about why a design was flagged and translate that into a different insertion strategy.

C. Iterative Structural Drift under Detector-Aware LLM Feedback

Fig. 6 illustrates the edge-level structural similarity between the original RTL design (ORI) and iteratively generated variants (A1–A4). Rather than monotonic divergence, heterogeneous structural trajectories emerge: in several cases the first refinement step induces noticeable deviation from ORI, followed by strong convergence among later variants (near-unity similarity between A2–A3 and A3–A4), as seen in GPT-4 on SRAM-HT3 (Fig. 6(e)) and Gemini-2.5Pro on AES-128-HT4 (Fig. 6(b)). Correlating with Table III, successful evasion tends to occur in a narrow regime of controlled structural drift — sufficient deviation from the detector's learned representations while preserving syntactic validity — whereas designs dominated by syntax errors correspond to unstable refinement paths where aggressive perturbations break RTL validity before evasion is achieved, and cases with minimal

structural change frequently remain detectable despite multiple attempts.

VII. Conclusions and Future Work

We presented TrojanGYM, an automated framework for curating HT insertions that expose detector blind spots. Starting from high-level HT specifications, an LLM-assisted agent suite realizes concrete insertions in RTL designs, and a gated loop enforces constraint-aware equivalence, syntax compliance, and GNN-based detection. This closes the gap between HT insertion and evaluation: detection is no longer a one-shot afterthought, but a signal that directly shapes which HTs are retained and how specifications are refined. In the case studies of SRAM, AES, UART, and RISC-V designs, TrojanGYM systematically generates diverse, functionally correct HTs that challenge state-of-the-art GNN detectors and reveal robustness gaps not visible in traditional TrustHub-style benchmarks.

Future work will expand both the property space and the feedback channels. For feedback, we plan to integrate testing-based HT detection frameworks. On the specification side, we plan to capture initial property-driven HT generation, trigger and payload behaviors, and constraints tied to performance or safety requirements.

References

[1] Y. Jin and Y. Makris, "Hardware trojan detection using path delay fingerprint," in *Proceedings of the IEEE International Workshop on Hardware-Oriented Security and Trust (HOST)*, 2008, pp. 51–57.

[2] A. Jayasena and P. Mishra, "Scalable detection of hardware trojans using ATPG-based activation of rare events," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 12, pp. 4450–4462, 2023.

[3] R. Yasaei, S.-Y. Yu, and M. A. Al Faruque, "Gnn4tj: Graph neural networks for hardware trojan detection at register transfer level," in *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2021, pp. 1504–1509.

[4] S. Yu, R. Yasaei, Q. Zhou, T. Nguyen, and M. A. Al Faruque, "HW2VEC: A graph learning tool for automating hardware security," in *2021 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*, 2021, pp. 13–23.

[5] Z. El Sayed, Z. Wang, H. Selmani, J. Knechtel, O. Sinanoglu, and L. Alrahis, "Graph neural networks for integrated circuit design, reliability, and security: Survey and tool," *ACM Computing Surveys*, vol. 58, no. 4, pp. 1–44, 2025.

[6] H. Salmani and M. Tehranipoor, "Trust-hub chip-level trojan benchmarks," https://trust-hub.org/\#/benchmarks/chip-level-trojan.

[7] H. Salmani, M. Tehranipoor, S. Sutikno, and F. Wijitrisnanto, "Trust-hub trojan benchmark for hardware trojan detection model creation using machine learning," 2022. [Online]. Available: https://dx.doi.org/10.21227/px6s-sm21

[8] Z. Pan, Z. Shu, L. Nguyen, and A. Alemayoh, "Sand: A self-supervised and adaptive nas-driven framework for hardware trojan detection," *arXiv preprint arXiv:2510.23643*, 2025.

[9] V. Gohil, H. Guo, S. Patnaik, and J. Rajendran, "Attrition: Attacking static hardware trojan detection techniques using reinforcement learning," in *Proceedings of the 2022 ACM SIGSAC conference on computer and communications security*, 2022, pp. 1275–1289.

[10] A. Sarihi, P. Jamieson, A. Patooghy, and A. Badawy, "Trojanforge: Adversarial hardware trojan examples with reinforcement learning," *arXiv preprint arXiv:2405.15184*, 2024.

[11] Z. Wang, M. Shao, A. Saha, R. Karri, J. Knechtel, M. Shafique, and O. Sinanoglu, "Netdetox: Adversarial and efficient evasion of hardware-security gnns via rl-llm orchestration," *arXiv preprint arXiv:2512.00119*, 2025.

[12] J. Cruz, P. Gaikwad, A. Nair, P. Chakraborty, and S. Bhunia, "Automatic hardware trojan insertion using machine learning," *arXiv preprint arXiv:2204.08580*, 2022.

[13] J. Cruz, Y. Huang, P. Mishra, and S. Bhunia, "An automated configurable trojan insertion framework for dynamic trust benchmarks," in *2018 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2018, pp. 1598–1603.

[14] V. Jyothi, P. Krishnamurthy, F. Khorrami, and R. Karri, "Taint: Tool for automated insertion of trojans," in *2017 IEEE International Conference on Computer Design (ICCD)*. IEEE, 2017, pp. 545–548.

[15] V. R. Surabhi, R. Sadhukhan, M. Raz, H. Pearce, P. Krishnamurthy, J. Trujillo, R. Karri, and F. Khorrami, "Feint: Automated framework for efficient insertion of templates/trojans into fpgas," *Information*, vol. 15, no. 7, p. 395, 2024.

[16] S. Thakur, B. Ahmad, H. Pearce, B. Tan, B. Dolan-Gavitt, R. Karri, and S. Garg, "Verigen: A large language model for verilog code generation," *ACM Transactions on Design Automation of Electronic Systems*, vol. 29, no. 3, pp. 1–31, 2024.

[17] Z. Wang, L. Alrahis, L. Mankali, J. Knechtel, and O. Sinanoglu, "Llms and the future of chip design: Unveiling security risks and building trust," in *2024 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*. IEEE, 2024, pp. 385–390.

[18] A. Saha, Z. Wang, P. B. Roy, J. Knechtel, O. Sinanoglu, and R. Karri, "Lockforge: Automating paper-to-code for logic locking with multi-agent reasoning llms," *arXiv preprint arXiv:2511.18531*, 2025.

[19] J. Blocklove, S. Garg, R. Karri, and H. Pearce, "Chip-chat: Challenges and opportunities in conversational hardware design," in *2023 ACM/IEEE 5th Workshop on Machine Learning for CAD (MLCAD)*. IEEE, 2023, pp. 1–6.

[20] Y. Liu, H. Zhang, Y. Zhou, Z. Shi, C. Xu, and Q. Xu, "Deeprtl2: A versatile model for rtl-related tasks," *arXiv preprint arXiv:2506.15697*, 2025.

[21] P. B. Roy, A. Saha, M. Alam, J. Knechtel, M. Maniatakos, O. Sinanoglu, and R. Karri, "Veritas: Deterministic verilog code synthesis from llm-generated conjunctive normal form," *arXiv preprint arXiv:2506.00005*, 2025.

[22] Z. Wang, M. Shao, J. Bhandari, L. Mankali, R. Karri, O. Sinanoglu, M. Shafique, and J. Knechtel, "Vericontaminated: Assessing llm-driven verilog coding for data contamination," *arXiv preprint arXiv:2503.13572*, 2025.

[23] A. Saha, P. B. Roy, J. Knechtel, R. Karri, O. Sinanoglu, and L. Alrahis, "Gllamor: Graph-based logic locking by large language models for enhanced robustness," in *2025 IEEE 43rd VLSI Test Symposium (VTS)*. IEEE, 2025, pp. 1–5.

[24] Z. Wang, M. Shao, M. Nabeel, P. B. Roy, L. Mankali, J. Bhandari, R. Karri, O. Sinanoglu, M. Shafique, and J. Knechtel, "Verileaky: Navigating ip protection vs utility in fine-tuning for llm-driven verilog coding," *arXiv preprint arXiv:2503.13116*, 2025.

[25] J. Knechtel, O. Sinanoglu, and R. Karri, "Llms for secure hardware design and related problems: Opportunities and challenges," *arXiv preprint arXiv:2605.10807*, 2026.

[26] Z. Wang, M. Shao, W. Fu, P. B. Roy, X. Guo, R. Karri, M. Shafique, J. Knechtel, and O. Sinanoglu, "Harmchip: Evaluating hardware security centric llm safety via jailbreak benchmarking," *arXiv preprint arXiv:2604.17093*, 2026.

[27] B. Perera, Z. Wang, W. Xiao, M. Nabeel, O. Sinanoglu, J. Knechtel, and R. Karri, "Llm4pqc - accurate and efficient synthesis of pqc cores by feedback-driven llms," *arXiv preprint arXiv:2602.09919*, 2026.

[28] J. Bhandari, R. Sadhukhan, P. Krishnamurthy, F. Khorrami, and R. Karri, "SENTAUR: Security enhanced trojan assessment using LLMs against undesirable revisions," *CoRR*, vol. abs/2407.12352, 2024. [Online]. Available: https://arxiv.org/abs/2407.12352

[29] V. T. Hayashi and W. V. Ruggiero, "Hardware trojan dataset of RISC-V and Web3 generated with ChatGPT-4," *Data*, vol. 9, no. 6, p. 82, 2024. [Online]. Available: https://doi.org/10.3390/data9060082

[30] M. O. Faruque, P. Jamieson, A. Patooghy, and A.-H. A. Badawy, "Unleashing ghost: An llm-powered framework for automated hardware trojan design," *Electronics*, vol. 14, no. 23, p. 4745, 2025.

[31] A. Sarihi, A. Patooghy, P. Jamieson, and A.-H. A. Badawy, "Trojan playground: a reinforcement learning framework for hardware trojan insertion and detection," *arXiv preprint arXiv:2305.09592*, 2023.

[32] R. Dai, Z. Liu, O. Arias, X. Guo, and T. Yavuz, "Dtjrtl: A configurable framework for automated hardware trojan insertion at rtl," in *Proceedings of the Great Lakes Symposium on VLSI 2024*, 2024, pp. 465–470.

[33] V. Gohil, S. Patnaik, D. Kalathil, and J. Rajendran, "{AttackGNN}:{Red-Teaming}{GNNs} in hardware security using

reinforcement learning," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 73–90.

[34] M. O. Faruque, P. Jamieson, A. Patooghy, and A.-H. A. Badawy, "Trojanwhisper: Evaluating pre-trained llms to detect and localize hardware trojans," *arXiv preprint arXiv:2412.07636*, 2024.

[35] S. Shankhdhar, A. Bharadwaj, V. Mishra, and S. Mittal, "Silentbite: A novel llm-based framework for automated hardware trojan insertion," in *2026 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 2026, pp. 3356–3360.

[36] H. Lashen, L. Alrahis, J. Knechtel, and O. Sinanoglu, "Trojansaint: Gate-level netlist sampling-based inductive learning for hardware trojan detection," in *2023 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 2023, pp. 1–5.

[37] L. L. Mankali, J. Bhandari, M. Alam, R. Karri, M. Maniatakos, O. Sinanoglu, and J. Knechtel, "Rtl-breaker: Assessing the security of llms against backdoor attacks on hdl code generation," in *2025 Design, Automation & Test in Europe Conference (DATE)*. IEEE, 2025, pp. 1–7.

[38] T. S. Sarkar, K. Arya, S. Chowdhury, U. Mandal, S. Shukla, S. Bhattacharya, and D. Mukhopadhyay, "Netlam: An automated llm framework to generate and evaluate stealthy hardware trojans," in *International Conference on Applied Cryptography and Network Security*. Springer, 2025, pp. 3–21.

[39] J. Chaudhuri, A. Chaudhuri, and K. Chakrabarty, "Latent: Llm-augmented trojan insertion and evaluation framework for analog netlist topologies," in *2025 IEEE International Conference on LLM-Aided Design (ICLAD)*, 2025, pp. 133–140.

[40] W. Xiao, Z. Wang, M. Shao, R. V. Hemadri, O. Sinanoglu, M. Shafique, J. Knechtel, S. Garg, and R. Karri, "Trojanloc: Llm-based framework for rtl trojan localization," *arXiv preprint arXiv:2512.00591*, 2025.